Journal of Intelligent Manufacturing (2024) 35:2593–2610
https://doi.org/10.1007/s10845-023-02161-w

Smart scheduling of dynamic job shop based on discrete event simulation and deep reinforcement learning

Ziqing Wang[1] · **Wenzhu Liao**[1]

Received: 23 November 2022 / Accepted: 2 June 2023 / Published online: 24 June 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023, corrected publication 2023

Abstract
In the era of Industry 4.0, production scheduling as a critical part of manufacturing system should be smarter. Smart scheduling agent is required to be real-time autonomous and possess the ability to face unforeseen and disruptive events. However, traditional methods lack adaptability and intelligence. Hence, this paper is devoted to proposing a smart approach based on proximal policy optimization (PPO) to solve dynamic job shop scheduling problem with random job arrivals. The PPO scheduling agent is trained based on an integration framework of discrete event simulation and deep reinforcement learning. Copies of trained agent can be linked with each machine for distributed control. Meanwhile, state features, actions and rewards are designed for scheduling at each decision point. Reward scaling are applied to improve the convergence performance. The numerical experiments are conducted on cases with different production configurations. The results show that PPO method can realize on-line decision making and provide better solution than dispatch rules and heuristics. It can achieve a balance between time and quality. Moreover, the trained model could also maintain certain performance even in untrained scenarios.

Keywords Job shop scheduling · Proximal policy optimization · Discrete event simulation · Deep reinforcement learning · Dynamic

Introduction

With increasing development and application of new generation information technology, such as cloud computing, Internet of Things (IoT), digital twin (DT) and artificial intelligence (AI), the fourth revolution in industry has been promoted, which is called Industry 4.0 (Qi & Tao, 2018). Industry 4.0 amounts to a paradigm change in manufacturing processes (Tao et al., 2017). The fusion of physical and the virtual world is made by cyber physical system. The 'things' (such as machines, products, people and data) are connected and interacted with each other to organize and conduct the industrial processes in new ways (Hermann et al., 2016). In manufacturing system, production scheduling could directly affect efficiency and cost. Thus, realizing smart scheduling is a fundamental way for enterprises to quickly respond to market changes, organize efficient production and meet diverse customer demand. Generally, four design principles are proposed for smart scheduling: automation, autonomy, real-time action capability and interoperability (Serrano-Ruiz et al., 2022). Responding to changing environment, smart scheduling also requires rapid adaptation and self-correction facing dynamic manufacturing system with random disturbance (Namjoshi & Rawat, 2022)。

Job shop scheduling problem (JSP) is viewed as a classic and difficult branch of production scheduling problem, which has been proved to be NP-hard (Gonzalez & Sahni, 1978). Its aim is to find the optimal sequence of jobs to be processed at different machines for objective optimization such as makespan or tardiness. So far, most studies on JSP have assumed that scheduling takes place in a static production environment, but the actual manufacturing system characterized by uncertainty and randomness inevitably need to handle dynamic events such as machine breakdown, job arrival or due date change. Therefore, it is of remarkable importance to develop a smart scheduling method for dynamic job shop scheduling problem (DJSP).

✉ Wenzhu Liao
liaowz@cqu.edu.cn

[1] Department of Engineering Management, Chongqing University, Chongqing 400044, China

Dynamic scheduling strategies can be classified into three categories: completely reactive scheduling, predictive–reactive scheduling and robust pro-active scheduling (Ouelhadj & Petrovic, 2008). The traditional approaches for these strategies face the dilemma of efficiency versus quality. Although metaheuristics acquire high solution quality, it need to be redesigned when problem structure changes. Hence, much more computation time would be taken especially facing frequent disturbance, which make it challenging for smart scheduling. Meanwhile, dispatch rules respond to dynamic events quickly in real-time at the expense of quality.

In recent years, due to the advantages of reinforcement learning (RL) in solving stochastic dynamic multi-stage sequential decision-making problems, its application and research in dynamic job shop scheduling has become a new hotspot. Scheduling agent (SA) based on RL can interact with the environment to learn optimal strategies independently, which satisfies smart scheduling design principles. Moreover, it reacts to dynamical changing environment completely in real-time to make on-line decisions. Hence, this paper is devoted to proposing a smart scheduling method considering SA realized by deep reinforcement learning (DRL) to solve DJSP with random job arrival with the aim of minimizing the mean tardiness.

In this paper, several contributions are given as follows:

(1) An interoperable environment for DRL training is provided. A DSJP model with random job arrivals based on discrete event simulation (DES) is built. Thereafter an integration framework of DRL and simulation model is proposed for real-time distributed scheduling.
(2) The definitions of states, actions, and rewards of Markov decision process (MDP) extracted from current simulation environment information are given. The state space consists of statistical values about job and the action space is based on dispatch rule. Hence, there will be no production scale limitations when using for making on-line scheduling decisions. In addition, because reward design can make a significant effect on models, the reward scale techniques are applied to improve the stability and efficiency of training.
(3) In the training phase, a value-based algorithm, proximal policy optimization (PPO), is chosen as schedule agent to learn policy. The numerical experiments show that the trained PPO has learned to select the most favorable dispatching rule to dynamically schedule jobs that wait in queue. It is more effective than dispatching rule and heuristics while can also maintain generalization in untrained situations.

Literature review

Traditional dynamic scheduling methods

Generally, dynamic scheduling strategies can be classified into three categories: completely reactive scheduling, predictive–reactive scheduling and robust pro-active scheduling (Ouelhadj & Petrovic, 2008). For completely reactive scheduling, it is also termed on-line scheduling, generating no firm schedule in advance and decisions are made in real-time driven by events. Dispatch rules are commonly used for on-line scheduling to determine the next job to be processed in a machine waiting queue. It is quick and easy to enforce but usually lead to poor solution quality. Moreover, it is impossible to identify any single rule as the best in all circumstances (Blackstone et al., 1982).

Predictive–reactive scheduling is the most common used and studied method, scheduling schemes are revised in response to real-time events. Compare with on-line scheduling, it maintains higher efficiency until rescheduling, which can seriously undermine solution performance. The rescheduling strategy need to be developed and the schedule plan are usually generated by meta-heuristics. For example, Lv et al. (2022) proposed a rescheduling mechanisms for both new job arrivals and machine tool breakdowns, and then applied the multi-objective simulated annealing to solve the rescheduling problem. To minimize the effects of disruption, a predictive schedule is pursued to satisfy performance requirements predictably in a dynamic environment. The main difficulty lies in the determination of robustness measures. Metaheuristics such as simulated annealing, genetic algorithm or particle swarm optimization are still popular in developing robust pro-active schedules (Duan & Wang, 2022; Souza et al., 2022).

However, it is difficult for aforementioned methods to strike the right balance between computational time and solution quality. These traditional methods face the dilemma of efficiency versus quality. Although metaheuristics acquire high solution quality, it is time-consuming and challenging for smart scheduling. Meanwhile, dispatch rules respond to dynamic events quickly in real-time at the expense of quality. Hence, due to the advantages of RL in solving stochastic dynamic multi-stage sequential decision-making problems, some research has been arisen by introducing RL in DJSP in recent years.

Scheduling by reinforcement learning

As dynamic scheduling problem can be model as a MDP that satisfies with the application premise of RL, RL agent decides the job in a queue to be processed. Early studies mainly use traditional tabular RL algorithms such as Q-learning (Aydin & Oztemel, 2000), the results show the feasibility of agents

based on RL in scheduling decisions. Bouazza et al. (2017) proposed a Q-learning approach to deal with dynamic flexible JSP with frequent jobs arrival. The Intelligent Product (IP) will choose machine selection rule and dispatch rule. Some studies also combine RL with other methods to improve performance. For DJSP with random job arrivals and machine breakdowns, Shahrabi et al. (2017) used Q-learning algorithm to obtain proper parameters for variable neighborhood search at rescheduling point. Wang (2020) established an improved weighted Q-learning algorithm based on clustering and dynamic search (WQ_CDS) to guide machine agents in scheduling policy choices in a dynamic environment.

RL methods store, and update estimated state-action value in lookup Q-table which are not available for continuous state space. The proposal of DRL (Mnih et al., 2015) solves the puzzle of handling high-dimensional and continuous input. Therefore, the application of DRL in the field of various production scheduling has gradually increased. Yang and Xu (2022) proposed a sysetem architecture of intelligent scheduling and reconfiguration in smart manufacturing and an advantage actor-critic (A2C) approach was used. Wang et al. (2022) solved JSP in a resource preemption environment with multi agent reinforcement learning. Yan et al. (2022) applied double-layer Q-learning algorithm (DLQL) to focuses on a digital twin-enabled integrated optimization problem of flexible JSP and flexible preventive maintenance.

For job shop scheduling problem, some studies build DRL frameworks based on graphs. Zhang et al. (2020) exploited the disjunctive graph representation of JSP and combine PPO with graph neural network to embed the states encountered during solving. A size-agnostic policy network was proposed to maintain generalization on large-scale instances. Lei et al. (2022) proposed a multi-pointer graph networks (MPGN) architecture and a training algorithm called multi-proximal policy optimization (multi-PPO) to two sub-policies for flexible JSP. They are both static environments. Jing et al. (2022) designed a centralized-learning decentralized-execution (CLDE) multi-agent reinforcement learning scheduling structure based on graph convolutional network (GCN). Although the scheduling performance in the dynamic scene is simply evaluated in the experimental part, it is not sufficient and the whole research framework is still based on the static environment.

Some studies trained agents directly in simulated environment. Luo (2020) designed six composite dispatching rules and seven generic state features for deep Q-learning (DQN) to solve flexible JSP with new job insertions. It is more like a kind of rescheduling method. Futhermore, to explore the performance of DRL agents more accurately in real-life manufacturing workshop environment. More studies executed in digital shop connected with actual shop. Li et al. (2022) developed the hybrid deep Q network (HDQN) to address a dynamic flexbile JSP with insufficient transportation resources. Zhang et al. (2022a) developed a distributed dueling deep Q network (D3QN) to realize real-time scheduling in cloud manufacturing to respond to dynamic and customized orders. Zhang et al. (2022b) applied PPO to establish the AI scheduler in a multi-agent manufacturing system and dynamic events were tested in the verfication experiments. However, these studies are excuted in fixed production configurations such as fixed machine number due to the limits of the environment.

Some research attempt to integrate RL with discrete event simulation (DES). DES could support for setting up an efficient, compatible and interoperable shop simulation environment to make up the DRL framework (Serrano-Ruiz et al., 2022). This greatly reduces the cost of experiments and enables adaptation to different production types. Lang et al. (2020) trained two DQN agents in connection with a DES model for the flexible JSP with integrated process planning. Kuhnle et al. (2021) addressed the design of RL to create an adaptive production control system by the real-world example of order dispatching in a complex job shop. Oh et al. (2022) proposed the scheduling method that combined the independent learners with the implicit quantile network (IQN) to solve the FJSP. For constant job arrivals, Liu et al. (2022) gave a distributed and hierarchical approach realized by double deep Q-network (DDQN) to make a real-time scheduling decision.

In summary, further research is needed on applying DRL for DJSP with random arrival of jobs. Some studies lack scalability due to environmental constraints. Building a simulation environment based on DES can improve the flexibility of the model, but few research has investigated the scenario of random arrival of jobs and lack analyze for the ability to generalize. Therefore, in this paper, PPO-based DJSP with random job arrivals is discussed to further supplement the relevant research. The scalability and generalization capabilities of the model in different production environments are concerned. The details about the process of developing and implementing DRL in dynamic scheduling with DES system are given. MDP model design is independent with problem size. Moreover, reward scaling is a hyperparameter in DRL which may lead to significant improvements, while has not been mentioned in previous literature. Hence, reward scaling is discussed in this paper to improve the quality of training and its impact is also analyzed.

DRL design

Scheduling agent

Each machine is equipped with a SA. When a machine needs to decide the job in the waiting queue to be processed

next, the required global and local information from other 'things' will be passed to its SA. The SA for each machine is only responsible for itself. This distributed approach can improve scheduling efficiency. As the scheduling process of all machines has the common nature, it is reasonable to set up a centralized SA to learn generic policy. And the copies of centralized SA can be used to make distributed decisions respectively for each machine, meanwhile empirical data collected by distributed SAs can help centralized SA to update network. Centralized learning can take advantage of global information to improve the accuracy and stability, which is the key part of implementing smart scheduling.

Reinforcement learning

The RL problem involves an agent exploring an unknown environment to achieve a goal. It is based on the hypothesis that all goals can be described by the maximization of expected cumulative reward. The agent must learn to sense and perturb the state of the environment using its actions to derive maximal reward. Generally, the theoretical framework for solving most RL problems is MDP defined by a tuple (S, A, P, R, γ). At each decision time t, the agent observes the environment state s_t, and select action a_t. The environment executes a_t to enter the next state s_{t+1} and feeds back a reward r_t to the agent, a number that tells it how good or bad the current world state is. When an agent follows a policy π that is the probability distribution of actions given a state, it generates the sequence of states, actions and rewards called the trajectory $\tau = (s_1, a_1, s_1 \ldots)$. Agents learn and update autonomously based on trajectories to find the optimal policy π_*.

Proximal policy optimization

The policy gradient methods aim at modeling and optimizing the parameterized policy directly (Sutton et al., 2000). Let π_θ denote a policy with parameters θ, and $J(\pi_\theta)$ denote the expected finite-horizon undiscounted return of the policy. The gradient of $J(\pi_\theta)$ is given as

$$\nabla_\theta J(\pi_\theta) = E_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta log\pi_\theta(a_t|s_t) A^{\pi_\theta}(s_t, a_t)\right] \quad (1)$$

where τ is a trajectory and A^{π_θ} is the advantage function for current policy that describes how much better or worse it is than other actions on average. The policy gradient algorithm works by updating policy parameters via stochastic gradient ascent on policy performance. There is

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta J\left(\pi_{\theta_k}\right) \quad (2)$$

PPO is an improved policy gradient method based on actor critic style. An actor controls how the agent behaves. And it updates the action distribution in the direction suggested by the critic. The critic estimates the value function, which measures how good the action taken is. Both actor and critic are parameterized with neural network. To reduce the impact of updating based on importance sampling, a special constraint on difference between the new and old policies is introduced. The constraint is expressed in terms of KL-divergence, a measure of distance between probability distributions. There are two primary variants of PPO: PPO-penalty and PPO-clip (Heess et al., 2017; Schulman et al., 2017). PPO-penalty penalizes the KL-divergence in the objective function and automatically adjusts the penalty coefficient over the course of training, so that it's scaled appropriately. PPO-clip does not have a KL-divergence, instead relies on specialized clipping in the objective function to remove incentives for the new policy to get far from the old policy. PPO-clip updates policies via Eq. (3, 4), shown as

$$\theta_{k+1} = argmax_\theta E_{s,a\sim\pi_{\theta_k}}[L(s, a, \theta_k, \theta)] \quad (3)$$

Here, L is given by

$$L(s, a, \theta_k, \theta) = min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t),\ g\left(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)\right)\right),$$
$$g(\epsilon, A) = \begin{cases} (1+\epsilon)A, & A \geq 0 \\ (1-\epsilon)A, & A < 0 \end{cases} \quad (4)$$

where ϵ is a hyperparameter for clipping.

Simulation model for DJSP

This paper studies on a DJSP with random job arrivals. There are n_{init} jobs $\left\{J_1 \cdots J_{n_{init}}\right\}$ and m machines $\{M_1 \cdots M_m\}$ in the workshop at the initial moment. Each job J_i consists of n_i operations $\left\{OP_{i1} \cdots OP_{in_i}\right\}$ processed on the machine in each order. n_{new} jobs arrive randomly during the workshop production. The goal of this DJSP is to schedule jobs in real-time to reduce the mean tardiness of all jobs. Here, some assumptions are given as follows:

(1) each machine can only process one job at a time;
(2) each operation can only be processed by one machine at the same time;
(3) each operation should be processed no preemptively without interruption;
(4) same priority between different jobs;
(5) sequential constraints between the operations of the same job.

For each job J_i with its arrival time A_i, its processing and sequence information can only be obtained after reaching

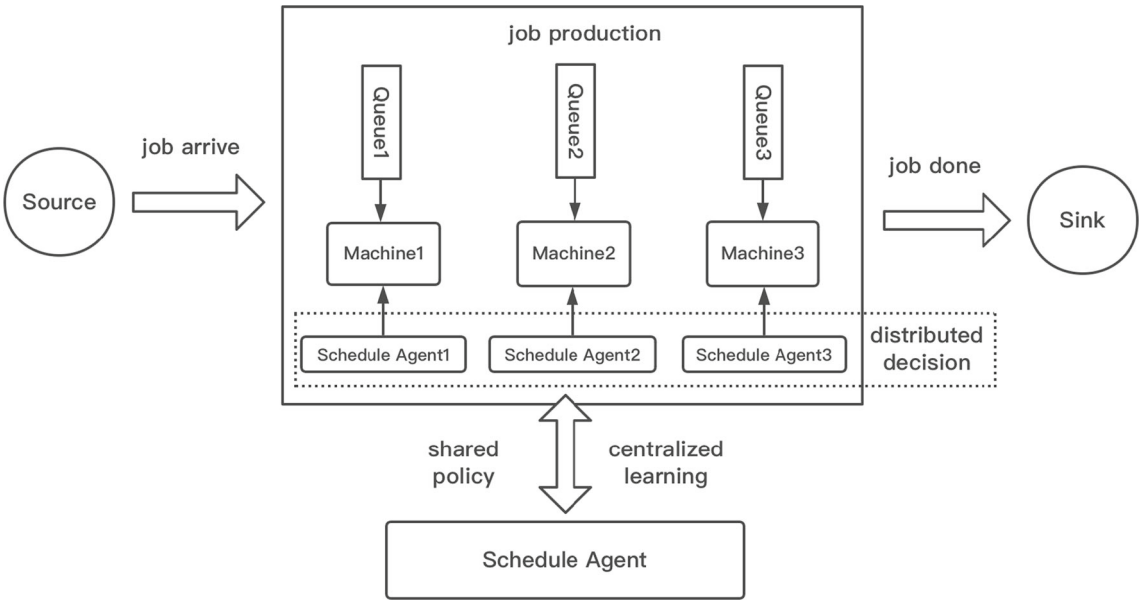

Fig. 1 An overview of the simulation process

system. Define D_i as the due date and C_i as the completion time of job J_i, the tardiness T_i of job J_i can be obtained as

$$T_i = \max(C_i - D_i, 0) \tag{5}$$

Hence, the optimization goal of this paper is the mean tardiness of all jobs T_{avg}, given as

$$T_{avg} = \sum_{i=1} \frac{T_i}{n_{init}+n_{new}} \tag{6}$$

In this paper, the proposed simulation model for DJSP is built with DES to set up an interoperable environment. Figure 1 shows an overview of the simulation process. The environment consists of 'Source', 'Queue', 'Machine', 'Sink' and 'Schedule Agent' with 'Job' flow as entities in the system. 'Source' constantly generates new 'Job' at random time intervals, then passes it into corresponding 'Queue' according to the sequence information. Each 'Machine' has a 'Queue' to put in all waiting jobs. After a 'Job' is processed on a 'Machine', if the number of completed operations does not correspond to the length of the operation sequence, the 'Job' flows to the next machine. Otherwise, the 'Job' computes its tardiness and enters to the 'Sink' which indicates leaving the system. The 'Sink' is used to store all the jobs that have been completed. When a 'Machine' is idle and the length of 'Queue' is not zero, a 'Job' selection decision needs to be made. If there is only one 'Job' in the 'Queue', the 'Job' can be directly selected for processing; if there are multiple jobs in the 'Queue', triggering the schedule event and the corresponding 'Schedule Agent' will be used to execute the decision and select the next 'Job'. All distributed schedule agents share the same policy with centralized 'Schedule Agent'. 'Schedule Agent' is realized by DRL, and it can achieve the goal of objective function during decision making through autonomous learning while interacting with the simulation environment. The following model and experiments are carried out in centralized SA.

Schedule agent based on DRL

Scheduling framework

In order to train the schedule agent, it is firstly to transform the scheduling process into a MDP model (i.e. it is required to design a complete set of states, actions and reward function). This paper provides a scheduling framework of DJSP with DRL in Fig. 2. When a machine is idle and there are multiple jobs in its waiting queue, the current simulation time T_{cur} is defined as a decision point t and the simulation is suspended. It means that simulation time and decision point are not synchronized. At step t, the state s_t of the job shop environment is passed to the schedule agent, and then it will feed back an action a_t to environment based on the state value and current policy π_θ. The action determines which job from the queue to be processed next. After the interactive decision-making process is completed, the simulation continues to run until the next decision point $t+1$ is encountered. Simulation ends if all jobs are scheduled. The above process is taken as one episode. Trajectories s_t, a_t, r_t, s_{t+1}, $a_{t+1}\cdots$ are collected through multiple episodes. The schedule agent will exploit trajectory information to update its own policy parameters.

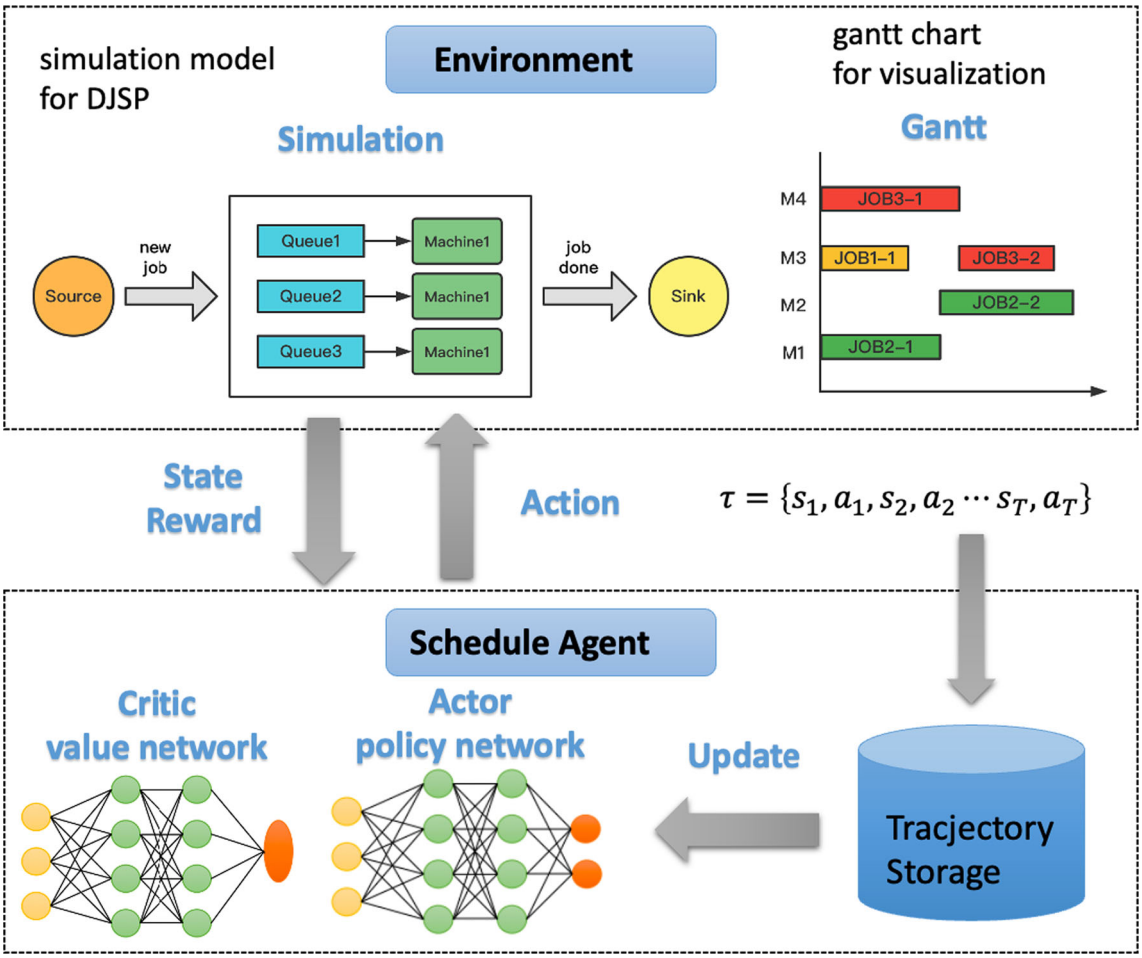

Fig. 2 The scheduling framework of DJSP with DRL

The training process and scheduling results can be displayed by Gantt charts.

MDP model

Action space

In this paper, eleven dispatch rules are selected as the action space, shown in Table 1. These dispatch rules are given as.

(1) the common basic dispatch rules SPT, LPT, FIFO, LOR and LWR;
(2) the rules related to the delivery date MS, EDD, CR and STOP;
(3) the combined rules LWRSPT and CRSPT.

Hence, the next job to be processed can be decided through the dispatch rules output by SA policy and the job information in the waiting queue at each decision point.

Table 1 Action space

Dispatch rule	Description
SPT	Select the job with the shortest imminent processing time
LPT	Select the job with the longest imminent processing time
FIFO	Select the job that arrives first
LWR	Select the job with the least work remaining
LOR	Select the job with the least operation remaining
MS	Select the job with minimum slack time
EDD	Select the job with earliest due date
CR	Select the job with smallest critical rate
STOP	Select the job with smallest slack time of operation
LWRSPT	Select the job with minimum sum of work remaining and imminent processing time
CRSPT	Select the job with minimum sum of critical rate and imminent processing time

State space

The state space is integrated with the setting of action space and reward function. Agent policy is a mapping of states to actions. Therefore, adding information related to the action space in the state features will make the learning process more efficient. The goal of agent is to maximize its cumulative reward, thus the states should also be able to reflect the reward value directly or indirectly. Moreover, it can describe the main features and changes of the scheduling environment, including global and local information (Han & Yang, 2020). The following information are all variables at the decision point t.

(1) System information
Define Job_{system} as the set of jobs still in the system at step t and Job_{queue} as the set of jobs in the queue to be scheduled. Through the simulation model, new jobs arrive in the system and completed jobs leave. Because the queues that need to make decisions are also different, the above two sets are dynamic.

(2) Job information
As the schedule agent selects a dispatch rule from the action space according to current policy, a job from the Job_{queue} will be assigned to the machine. The following related information for each job $J_i \in Job_{queue}$ is thus given.

(a) TTD_i: the remaining time of job J_i to the due date D_i
The due date tightness factor f_i is an indicator of job urgency. PT_{ij} denotes the processing time of operation OP_{ij}. For a job with arriving time A_i, its due date can be calculated in Eq. (7). TTD_i is the difference between its due date and current time calculated as Eq. (8). Every time a job needs to be selected from the queue for processing whether schedule agent is required, updating the information of all jobs in Job_{system}.

$$D_i = A_i + f_i \cdot \sum_{j=1}^{n_i} PT_{ij} \tag{7}$$

$$TTD_i = D_i - T_{cur} \tag{8}$$

(b) ROP_i: the remaining operation number of job J_i
Let OP_{icur} denotes the current operation index, ROP_i can be obtained as

$$ROP_i = n_i - OP_{icur} \tag{9}$$

(c) RPT_i: the remaining processing time of job J_i
The remaining processing time of job J_i is given as

$$RPT_i = \sum_{j=OP_{icur}}^{n_i} PT_{ij} \tag{10}$$

(d) NPT_i: the processing time of next operation of job J_i
The processing time of next operation of job J_i is given as

$$NPT_i = PT_{i(OP_{icur})} \tag{11}$$

(e) ST_i: the slack time of job J_i
The slack time of job J_i is between its TTD_i and RPT_i, calculated as

$$\mathrm{ST_i} = TTD_i - RPT_i \tag{12}$$

(3) Queue information
After the job is processed, it will be transferred to its next queue. Define NTQ_i as the number of waiting jobs in the next queue and WTQ_i as total processing time of all waiting jobs in the next queue.

(4) State space
The above features are in the form of a set with a length of Job_{queue}. Besides the number of jobs in Job_{system} and Job_{queue}, several statistical characteristics of these features are selected: mean, standard deviation, maximum value, and minimum value. All of them make up the state space, shown in Table 2. The dimension of state space is $7 \times 4 + 2 = 30$. The value of all variables for the end is set to zero. It should be noted that some original value ranges are too different, for example $ROP(t)$ and $RPT(t)$ during the training process. To improve the stability of neural network training, the running mean and standard deviation of each variable are calculated dynamically. The state is input into network after normalized.

Reward function

Reward function is the specific and digitized goal, which determines the direction and efficiency of the agent's learning. The agent would have corresponding feedback information after performing an action in a certain state. However, in JSP, the tardiness will only be obtained when a job has been delayed. There is no corresponding reward in the early training stage, thus other quantitative indicator needs to be used instead of the objective value. At step t, the estimated tardiness $T_{i(es)}(t)$ of job J_i depends on estimated completion time $C_{i(es)}$. Assume $C_{i(es)} = now + RPT_i$, by converting, the slack time can be used as a reward proxy

Table 2 State space

Features	State	Description
$TTD(t)$		Time till due date
$ROP(t)$	Max	Number of remaining operations
$RPT(t)$	Min	Remaining processing time
$NPT(t)$	Mean	Processing time of next operation
$ST(t)$	Std	Slack time
$NTQ(t)$		Number of waiting jobs in the next queue
$WTQ(t)$		Processing time of waiting jobs in the next queue
Job_{system}	Number	Number of jobs in the system
Job_{queue}		Number of jobs in the queue

instead of tardiness, shown as

$$\begin{aligned} T_{i(es)}(t) &= \max\left(C_{i(es)} - D_i, 0\right) \\ &\approx \max(now + RPT_i - D_i, 0) \\ &\approx \max(RPT_i - \mathrm{TTD_i}, 0) \\ &\approx \max(-\mathrm{ST}_i, 0) \end{aligned} \tag{13}$$

The reward is set as the difference between the total slack time of step t and time $t+1$, calculated by Eq. (14). It will not be obtained until the next decision point. Since from step t to step $t+1$, new jobs may arrive or jobs may leave the system, the set of jobs for calculating the slack time is based on $\left\{\mathrm{ST_i} | i \in Job_{system}(t)\right\}$. β is the reward scaling factor to adjust reward range which affects the value function of states. It is set to be 0.05 during training. There is

$$R(t) = \sum_{i \in Job_{system}(t)} (\mathrm{ST}_i(t+1) - \mathrm{ST}_i(t)) \cdot \beta \tag{14}$$

Algorithm

As this paper used PPO-clip as schedule agent for training, the description of PPO-clip algorithm for DJSP based on DES is given as follows. It is known that estimating advantage function $A^{\pi_{\theta_k}}(s_t, a_t)$ is a critical part for PPO implementation. This paper adopts the generalized advantage estimation (GAE) algorithm (Schulman et al., 2015) which is one of the most effective approaches. It can exponentially weight multi-step estimates to balance variance and bias. The GAE function in this paper is defined as

$$\widehat{A}_t^{GAE(\gamma,\lambda)} = \sum_{l=0}^{\infty} \gamma\lambda^l \delta_{t+1}^V \tag{15}$$

where $\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$ is the temporal-difference error.

For each training episode, reset environment and state at start. The simulation runs until a decision point t occurs, the current state s_t about system and Job_{queue} is input into PPO actor network. A SoftMax probability distribution of eleven dispatch rules will be output based on current policy π_{old}. The agent samples a random action a_t according to the distribution and returns it to the environment. Hence, one dispatch rule is selected and which job in queue should be processed is known. The simulation continues until next decision point $t+1$ when renewing the information of all jobs in the system again. Reward r_t of action a_t based on the slack time can be calculated now and fed back to PPO.

Action probability $p(a_t)$ and state value function $V(s_t)$ obtained by critic network are recorded at each step. The trajectory $(s_t, a_t, r_t, V(s_t), p(a_t))$ is kept in storage M_{old}. After a fixed number of steps, the batch advantage function $\widehat{A}$ with GAE is firstly calculated. Then, the trajectories are divided into mini batches. Meanwhile, the actor and critic network parameters are updated repeatedly by using these data. New trajectories will be exploited for learning at next time. Afterward schedule agent based on PPO interacts with the environment by new policy π_{new}. As this paper uses PPO-clip as schedule agent for training, the description of PPO-clip algorithm for DJSP based on DES is given as follows.

Algorithm 1. PPO-clip algorithm for DJSP based on DES.

Initialize Actor network parameters θ_0 (policy network)

Initialize Critic network parameters ϕ_0 (value network)

For episode = 1, M do:

Initialize simulation environment and reset state s_1

While simulation run:

If now is decision point t (machine is idle and there are at least 2 jobs waiting in queue):

Obtain action a_t and $p(a_t)$ based on current policy $\pi_{old} = \pi(\theta_{old})$ with current state s_t as input

Obtain value $V(s_t)$ from critic network ϕ_{old} with current state s_t as input

Execute action a_t and choose a job for machine from queue

Get reward r_t and next state s_{t+1} from environment

Put trajectory $\tau_t = (s_t, a_t, r_t, V(s_t), p(a_t))$ into storage M_{old}

Set $s_t = s_{t+1}$

If M_{old} = batch size:

For n=1, N (repeat times):

Using GAE compute batch advantage estimates $\hat{A}$

For each mini batch:

Compute new returns R_{mini} and new value V_{mini}

Update the policy by maximizing the PPO-Clip objective by stochastic gradient ascent with Adam

$$\theta_{k+1} = argmax_\theta \frac{1}{|M_{old}|T} \sum_{\tau \in M_{old}} \sum_{t=0}^{T} \min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), g\left(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)\right) \right)$$

Fit value function by regression on mean-squared error by stochastic gradient descent algorithm with Adam

$$\phi_{k+1} = argmax_\phi \frac{1}{|M_{old}|T} \sum_{\tau \in M_{old}} \sum_{t=0}^{T} (V_{mini} - R_{mini})^2$$

Clear M_{old}

End for

Table 3 Parameter setting of production configuration

Parameter	Value
Number of initial jobs (n_{init})	10
Number of machines (m)	{8, 10, 15}
Number of operations in a job (op_nb)	$Randint[1, m]$
Number of newly arrived jobs (n_{new})	{20, 50, 100}
Average value of exponential distribution between two successive job arrivals (E_{new})	{20, 30, 40}
Processing time of an operation (PT_{ij})	$U[10, 50]$
Due date tightness factor (f_i)	{1, 1.2, 1.5}

Table 4 Parameters for PPO

Type	Hyperparameters	Value
DNN	Hidden layers	3
	Hidden nodes	512
	Gradient clip	0.5
DRL	Train episodes	5000
	Batch size	512
	Mini batch size	64
	Update epochs	10
	Gamma	0.995
	GAE lambda	0.95
	Policy epsilon	0.2
	Actor learning rate	3e-5
	Critic learning rate	3e-5
	Entropy coefficient	0.01
	GAE lambda	0.95

Experiments

Environment

The established deep learning framework PyTorch used in this paper is developed based on the python language. Considering the availability of suitable interfaces with the widely used commercial simulation tools such as Simulink or AnyLogic, the simulation is implemented based on Python-based DES library SimPy. The proposed PPO is compiled with python3.9 and implemented in PyCharm on a PC with Apple M1, 8 GB RAM, macOS Monterey system.

It is assumed that 10 jobs exist in the job shop floor at the very beginning. The arrival of subsequent new jobs follows a Poisson distribution, hence the interarrival time between two successive new job is subjected to exponential distribution with average value E_{new} (Chang et al., 2022). The related parameter settings of production configuration are shown in Table 3. The number of operations per job does not exceed the number of machines. Processing time of each operation PT_{ij} is satisfied with a uniform distribution. Due date tightness factor f_i of job J_i is randomly selected from the set {1, 1.2, 1.5}.

Training

The PPO is trained in a simulated job shop environment with 8 machines and 20 new jobs, and average value of interarrival time E_{new} is 20. Here, Actor and Critic are separate deep neural networks both consisting of five fully connected layers with three hidden layers except for the input and output layers. Meanwhile, tanh function is used as the activation function and Adam is adopted as the optimizer. By clipping gradient norm of parameters, the problem of exploding or vanishing gradients can be avoided. The parameter settings of PPO are listed in Table 4.

To demonstrate that the reward function setting in this paper enable PPO to learn towards smaller mean tardiness, the corresponding T_{avg} and episode reward of each dispatch rule in the action space are recorded in the end. It can be obtained that the calculated Pearson correlation coefficient is -0.998, which confirms that the reward function is consistent with the optimization objective. The results of episode reward and mean tardiness obtained by PPO-based smart SA during 5000 training episodes are presented in Fig. 3. It can be found that episode reward increases significantly from initial value almost -200 to -140 after 3000 episodes and eventually converges. In the meantime, the mean tardiness T_{avg} decreases dramatically from 108 to 67.32.

The corresponding T_{avg} list is acquired by regarding each dispatch rule as the fixed action at every decision point t. Among the eleven rules, the optimal one is LWR. Its corresponding T_{avg} is 84.8, and the average T_{avg} of all rules is 102.34. It shows that trained PPO outperforms than any dispatch rule of action space. Moreover, the trained model is tested by 20 times in the same environment, then the number of times each action is selected is recorded. The proportions of selected actions are shown in Fig. 4. It validates that the PPO-based schedule agent can be trained to learn to choose different actions in different states while achieving overall optimization. Gantt charts of simulated environment by PPO and FIFO are shown in Figs. 5 and 6, respectively.

Reward scale

Learning of DRL algorithms is oriented by the reward, and reward scaling can have a large effect on the results (Henderson et al., 2018). Therefore, in order to explore the impact of different β values on the performance of PPO, five cases are tested for 5000 training iterations in this paper, where $\beta \in \{0.001, 0.005, 0.01, 0.05, 0.1\}$. The original episode

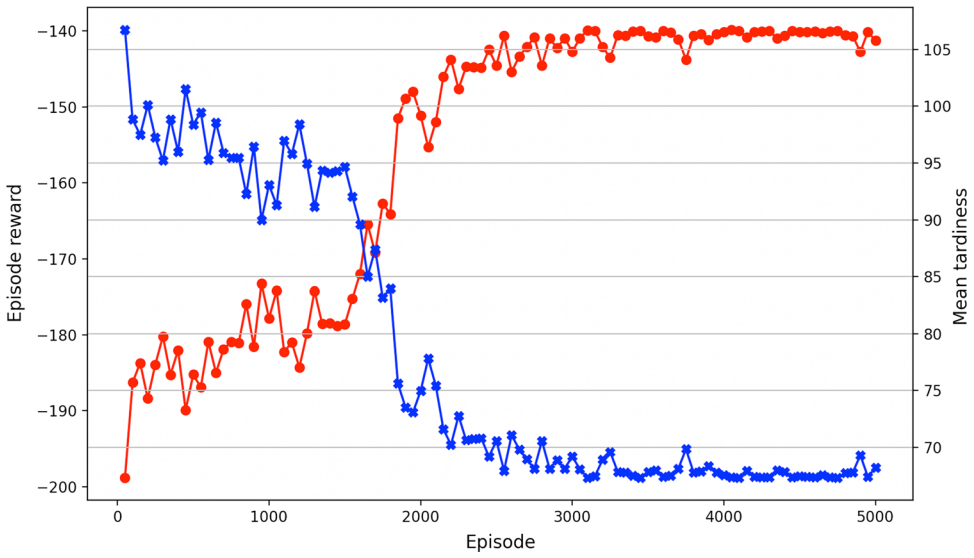

Fig. 3 The training results by PPO

rewards obtained by setting different reward scale factors have different ranges. For comparison, the results are scaled to the same scale with $\beta = 0.05$ as the benchmark. In can be seen that in Fig. 7, when $\beta \in \{0.005, 0.01, 0.05\}$, the final performances of training are not much different, and the convergence trends all appear when it is close to 3000 episodes. PPO learns slightly faster with $\beta = 0.05$. However, if $\beta = 0.1$ or $\beta = 0.001$, the training effect is significantly worse, and the reward at convergence is much lower than it with $\beta = 0.05$. Hence, according to the results analysis above, it is found that the reward scaling could maintain a good effect within a certain range. And too low or too high β value would destructive the convergence speed and final performance.

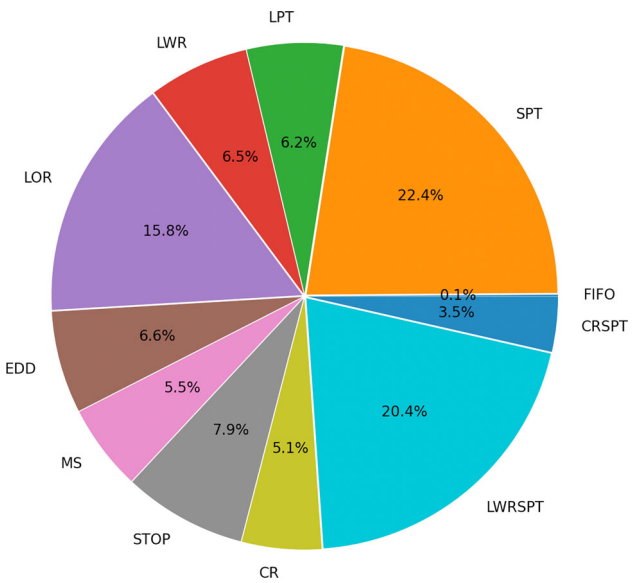

Fig. 4 The proportions of selected actions by trained PPO at each decision point

Comparison

Comparison with dispatch rules

To verify the generalization ability of trained PPO, 27 cases are randomly generated based on different parameter combinations of (E_{new}, m, n_{new}) in this paper. The state is first standardized and then input into the network when training the PPO model. Hence, before testing each case, the mean and variance of state variables in this case are obtained by random walk 300 times. Comparing the test results with dispatch rules used in this paper, the data about T_{avg} are shown in Table 5. Define AT_{avg} is the average value of T_{avg} from all cases. The results of AT_{avg} are given in Fig. 8. Based on a comparison of AT_{avg} by PPO and other dispatch rules, it can be found that PPO can also achieve better performance than dispatch rules in new cases without retraining. Indicating that the scheduling agent has a certain degree of generalization. The AT_{avg} of PPO is 83.37, while its value range of rules is [92.61, 139.94].

Comparison with heuristics

To further analyze the effectiveness of the proposed PPO method in this paper, it is compared with three methods including meta-heuristics and hyper-heuristics. Mean tardiness and computation time are chosen as metrics to explore performance in time and quality. As CRSPT performs best in the previous experimental analysis, it is chosen as the baseline rule. The three comparison methods are described below.

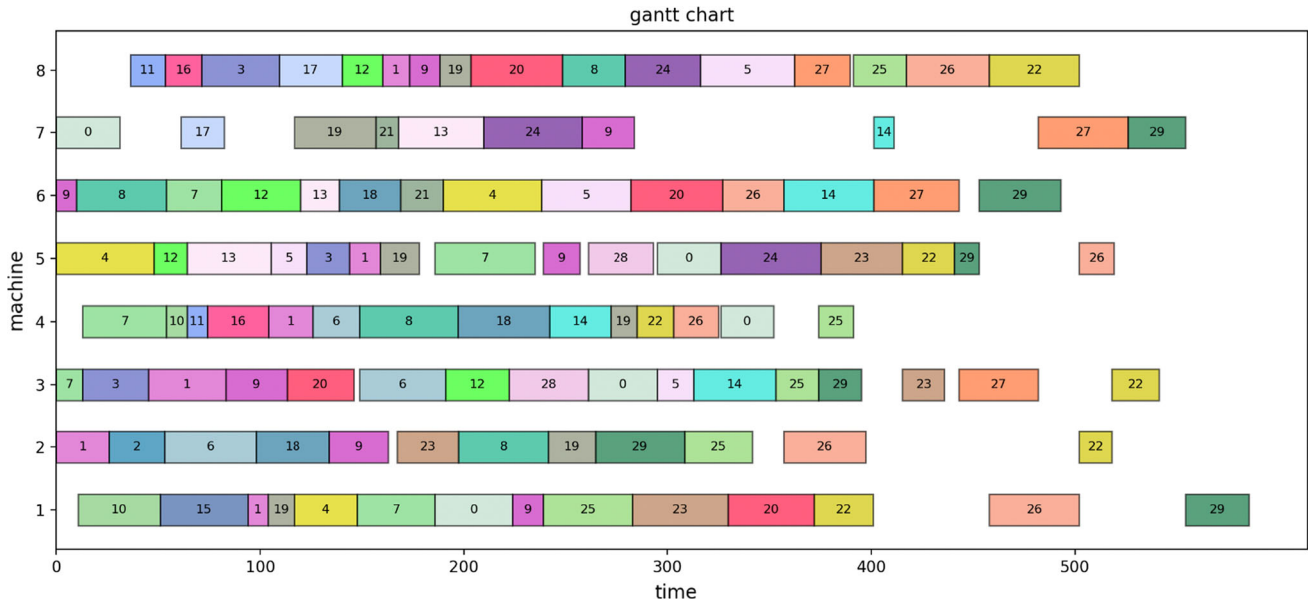

Fig. 5 Gantt chart of simulated environment by PPO

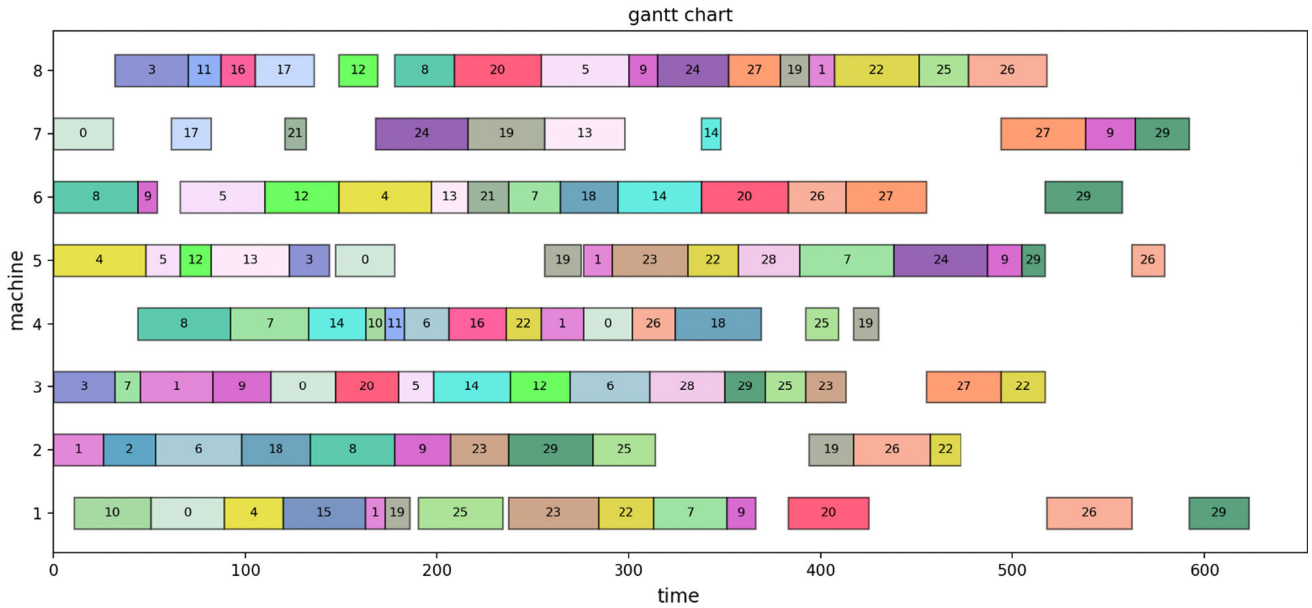

Fig. 6 Gantt chart of simulated environment by FIFO

(1) Genetic Algorithm (GA): GA is a kind of classic meta-heuristics that has been widely used in job shop scheduling. Since the problem covered in this paper considers random job arrivals, rescheduling is required for adjusting the schedule plan. The rescheduling mechanisms is followed by (Lv et al., 2022). The system classifies the task set that belongs to jobs into three subsets, then get new schedule solution with GA at each rescheduling point. While new jobs keep arriving, rescheduling will be trigged when a total of five new jobs waiting for assignment.

(2) Genetic Programing (GP): GP can be used for discovering new dispatching rules, which are naturally represented as GP trees. The primitive set is followed by (Hunt et al., 2014).The model is trained based on the DEAP framework.

(3) GA-based Hyper Heuristic (GAHH): the GAHH framework is followed by (Akarsu & Küçükdeniz, 2022).The approach is expressed through two modules: hyper heuristic module and low-level heuristics module (LLH). LLH includes dispatch rules to generate a solution in specific problem. Hyper heuristic module is

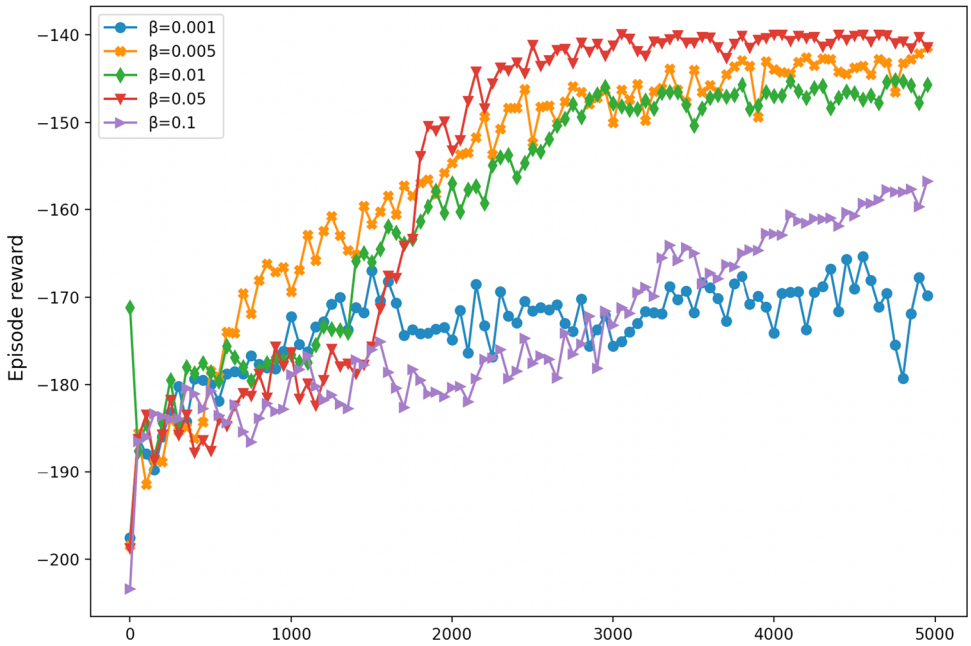

Fig. 7 The performance of PPO under different β values

based on GA. The individual represents the sequence of low-level heuristics. The model is trained based on the DEAP framework.

Firstly, assuming that the dynamic information in the shop is known, GP and GAHH can be retrained in each specific problem to find the optimal solution. The results of scheduling by five methods are shown in Table 6. GA takes significantly more time than other methods to obtain scheduling solutions. Due to the frequent arrival of jobs, multiple rescheduling has been carried out in the whole process which increases the computational effort. The calculation time of GP and GAHH is similar and significantly less than that of GA. Because both CRSPT and PPO can see a real-time strategy and only the relevant information of the jobs in the queue needs to be input, the decisions can be made very quickly. In all cases, PPO outperforms other methods in 20 cases, accounting for 74% with AT_{avg} value of 83.37, followed by 15% of GP with AT_{avg} value of 87.16 and 13% of GAHH with AT_{avg} value of 89.49. This demonstrates the effectiveness and superiority of the proposed approach in this paper in tackling the dynamic job shop scheduling problem with random job arrivals.

Moreover, in real-world shop floor environments, new job information may be not known in advance. Hence, the scheduling system is required to generate decisions in response to dynamically changing conditions, with a requirement for rapid response times. It practically relies on methods that require repeated training iterations such to converge to a satisfactory solution, which is a challenge. Based on the consideration of practical constraint, the performance of various methods in real-time scheduling is further tested. The heuristic solutions generated by GP and GAHH in the same trained environment with PPO are regarded as a kind of dispatch rule to make real-time decisions. GA lack the ability to cope with dynamic job arrival and cannot be applied to real-time scheduling (Liu et al., 2022). The data in Table 6 also confirmed this conclusion, thus GA are not used for comparison in this experimental analysis. The cases are the same as the previous configurations, but with different specific job information. According to the results in Table 7, PPO outperforms other methods in all cases and its AT_{avg} is equal to 83.90, better than 90.41 of CRSPT, 93.74 of GP and 100.78 of GAHH.

Comparing with problem-specific algorithms GAHH and GP, PPO shows remarkable scalability and generalization ability under different scale problems. This suggests that the proposed PPO-based agent could be a more versatile and robust solution for real-time scheduling problems, particularly when dealing with uncertain and unpredictable events such as random arrival of jobs.

Conclusion

In the context of Industry 4.0, the scheduling system is required to be smarter, which is characterized by autonomy learning, real-time reaction, and ability to handle with the dynamic production environment. In this paper, a smart scheduling method based on DRL is implemented. The agent is trained by interacting with a dynamic job shop scheduling environment, which is built with discrete event simulation. Random job arrivals are considered as the dynamic event.

Table 5 The mean tardiness obtained by PPO and dispatch rules

E_{new}	m	n_{new}	FIFO	SPT	LPT	LWR	LOR	EDD	MS	STOP	CR	LWRSPT	CRSPT	PPO
20	8	20	125.08	98.52	130.52	84.8	99.58	88.6	116.41	99.28	97.88	88.89	96.26	67.33
		50	119.35	84.75	134.48	87.94	95.87	92.32	108.06	100.09	101	79.71	82.84	69.57
		100	155.69	116.12	183.23	134.66	142.49	140.7	150.42	143.34	137.94	119.5	119.24	113.2
30	8	20	80.62	59.21	84.14	64.76	72.42	64.63	67.09	63.67	63.41	57.99	59.21	48.83
		50	52.12	37.47	55	40.8	45.14	40.67	44.13	41.73	41.07	39.08	37.38	31.29
		100	63.93	50.71	70.95	58.51	62.05	54.55	60.96	60.35	57.19	58.81	49.92	49.09
40	8	20	57.49	46.54	75.14	44.75	50.66	51.89	56.37	50.69	51.13	44.96	44.67	33.63
		50	29.46	21.62	42.75	24.05	26.8	29.5	28.53	24.87	25.09	25.16	20.7	15.13
		100	38.05	25.47	46.12	31.79	34.45	34.49	31.58	27.72	28.84	32.37	23.94	20.38
20	10	20	241.16	190.06	261.02	188.17	203.8	198.36	202.96	189.57	184.31	184.26	180.96	170.04
		50	240.48	184.03	293.83	190.54	209.54	204.11	204.4	200.1	200.96	182.25	178.13	186.39
		100	241.24	186.67	345.32	206.07	213.32	223.83	223.23	217.12	208.59	186.8	180.84	188.43
30	10	20	193.06	160.75	219.45	153.32	160.52	156.96	183.09	167.94	164.58	152.21	157.22	144.81
		50	130.9	110.86	155.13	106.69	114.07	110.21	123.41	116.02	116.09	105.85	113.18	102.82
		100	90.37	80.77	106.65	81.21	83.16	78.48	84.41	79.72	82.64	75.7	79.32	72.85
40	10	20	158.84	136.75	182.53	123.34	135.81	140.5	147.92	126.47	127.96	121.5	132.1	115.96
		50	90.29	74.45	109.08	72.61	79.18	78.02	82.37	71.44	72.18	70.5	70.84	64.65
		100	54.85	45.5	68.37	47.35	49.18	46.62	49.07	42.9	42.26	42.84	43.53	41.24
20	15	20	167.42	144.17	193.49	132.94	135.69	136.57	187.68	156.04	164.66	131.56	141.43	116.55
		50	156.61	125.11	186.31	125.58	124.9	140.14	195.16	159.02	170.67	123.34	128.26	111
		100	178.14	137.33	212.97	155.03	148.57	165.97	193.67	182.34	182.4	149	135.15	127.46
30	15	20	143.47	118.16	168.74	118.91	120.94	115.4	147.5	126.4	127.03	124.12	117.16	103.86
		50	86.01	72.37	119.59	82.98	78.8	79.91	90.42	76.88	77.99	87.65	73.36	61.02
		100	67.28	53.82	90.53	62.04	60.59	63.55	64.72	54.69	55.71	69.59	54.41	50
40	15	20	122.32	100.09	127.56	92.01	101.62	84.95	102.71	97.96	101.71	96.04	102.42	86.58
		50	61.05	51.43	68.78	47.83	48.24	41.33	53.03	45.45	47	48.51	48.31	37.55
		100	36.35	31.25	46.57	27.1	29.81	24.63	33.21	28.54	28.29	31.09	29.56	21.45

The settings of state and reward are very critical while designing a MDP model, however the characteristic information of dynamic production environment is very complex. Therefore, owing to DRL algorithms' strong ability to perceive and fit high-dimensional input, this paper develops 30 abstracted features about system and queue at the decision point. Then, in order to obtain the appropriate reward scale factor, 5 cases with different values are experimented. The results illustrate that convergence performance could be maintained well if the scaling of reward is in a certain range.

The proposed SA in this paper can enforce stable learning and training curve eventually converges. The mean tardiness of jobs as the optimization objective is also decreasing gradually. And the experiments show that this PPO-based SA can achieve better quality solution than each dispatch rule in the action space after training . In addition, comparaing with three other heuristic, this method also possess the potential competitiveness while generalizing to other unseen scenarios. In actual manufacturing system, the distribution of data in a workshop tends to be regular. More accurate statistical values of states can be obtained. Besides the features in the state space are not related to the production configuration. Hence, it is feasible to training the SA in other environments. In this application, each machine can be connected to a copied agent from trained SA to achieve distributed control, which could reduce the computing burden and increase efficiency. This proposed PPO-based method can well support the design requirements of smart scheduling. It can achieve high quality and responsiveness simultaneously in a dynamic production environment. Furthermore, the experiential knowledge could be used to learn for making more efficient online decisions.

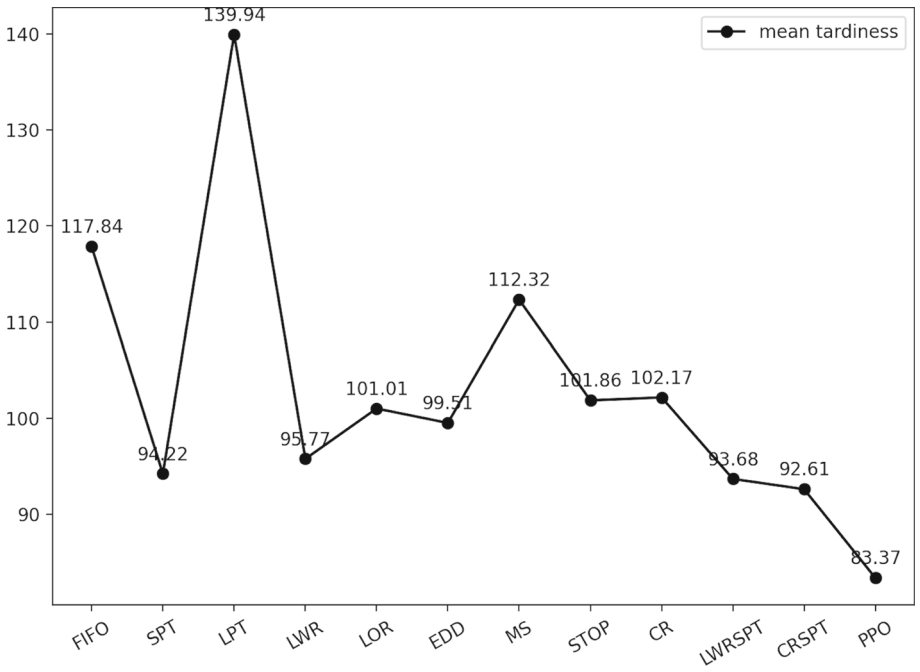

Fig. 8 The mean tardiness obtained by PPO and dispatch rules

Table 6 Mean tardiness and computation time of five methods in known cases

E_{new}	m	n_{new}	CRSPT		GP		GAHH		GA		PPO	
			Value	Time	Value	Time	Value	Time	Value	Time	Value	Time
20	8	20	96.26	0.05	71.29	6.62	82.13	8.07	100.05	55.55	67.33	0.06
		50	82.84	0.08	77.85	9.54	84.47	22.11	96.82	212.48	69.57	0.13
		100	119.24	0.15	118.79	19.39	121.44	35.68	146.42	721.46	113.2	0.26
30	8	20	59.21	0.05	52.35	6.43	50.69	7.98	69.54	51.63	48.83	0.06
		50	37.38	0.08	34.77	9.9	34.56	22.09	59.78	189.54	31.29	0.1
		100	49.92	0.15	56.32	21.78	47.48	36.73	77.37	630.19	49.09	0.19
40	8	20	44.67	0.05	39.87	6.94	34.34	8.03	64.66	50.17	33.63	0.06
		50	20.7	0.07	17.81	10.68	19.75	22.77	48.81	187.52	15.13	0.08
		100	23.94	0.14	23.33	20.33	23.14	37.14	51.72	608.48	20.38	0.16
20	10	20	180.96	0.06	192.27	9.01	177.66	9.48	255.21	71.71	170.04	0.09
		50	178.13	0.11	181.92	13.93	189.79	23.48	261.52	298.56	186.39	0.16
		100	180.84	0.22	175.94	22.77	205.14	37.82	273.14	1001.01	188.43	0.32
30	10	20	157.22	0.06	152.37	7.53	148.75	10.93	197.79	69.93	144.81	0.08
		50	113.18	0.1	104.63	10.16	100.54	23.35	144.65	260.37	102.82	0.14
		100	79.32	0.17	73.63	20	77.73	37.81	113.37	810.89	72.85	0.27
40	10	20	132.1	0.06	123.88	6.36	122.17	9.65	149.51	69.96	115.96	0.07
		50	70.84	0.09	64.07	9.86	70.3	23.87	94.91	247.34	64.65	0.11
		100	43.53	0.15	42.4	19.21	41.96	38.99	71.09	766.65	41.24	0.17
20	15	20	141.43	0.06	125.23	6.81	131.7	9.01	266.58	94.91	116.55	0.08
		50	128.26	0.13	117.47	10.02	123.45	24.94	255.71	422.73	111	0.17
		100	135.15	0.22	128.99	20.38	139.81	41.12	255.02	1379.89	127.46	0.33

Table 6 (continued)

E_{new}	m	n_{new}	CRSPT		GP		GAHH		GA		PPO	
			Value	Time	Value	Time	Value	Time	Value	Time	Value	Time
30	15	20	117.16	0.06	113.44	6.35	114.319	8.51	209.08	96.11	103.86	0.08
		50	73.36	0.1	65.22	11.7	70.49	23.79	151.99	375.46	61.02	0.13
		100	54.41	0.16	50.06	21.28	48.83	40.22	124.82	1176.90	50	0.23
40	15	20	102.42	0.05	84.94	6.71	90.37	8.69	185.71	96.95	86.58	0.07
		50	48.31	0.09	41.58	9.34	39.65	25.11	122.11	354.06	37.55	0.11
		100	29.56	0.15	23.08	20.99	25.78	43.02	97.78	1067.21	21.45	0.21

Table 7 Mean tardiness and computation time of five methods in unknown cases

E_{new}	m	n_{new}	CRSPT		GP		GAHH		PPO	
			Value	Time	Value	Time	Value	Time	Value	Time
20	8	20	94.50	0.05	81.38	0.05	96.36	0.04	70.81	0.07
		50	86.42	0.08	86.29	0.06	100.3	0.06	75.13	0.14
		100	120.16	0.16	138.9	0.11	140.1	0.11	118.2	0.26
30	8	20	56.23	0.04	60.68	0.03	66.95	0.03	53.49	0.06
		50	43.88	0.08	43.88	0.06	43.35	0.06	35.1	0.11
		100	52.64	0.14	62.18	0.11	57.87	0.11	47.69	0.18
40	8	20	39.62	0.04	37.55	0.03	31.69	0.03	25.01	0.05
		50	21.71	0.08	21.1	0.06	18.99	0.06	14.6	0.09
		100	24.36	0.17	27.81	0.10	27.89	0.10	23	0.16
20	10	20	182.15	0.04	188.3	0.03	194.3	0.03	174.6	0.08
		50	178.88	0.08	187	0.06	193.7	0.06	178.25	0.16
		100	179.66	0.14	201.4	0.10	220.8	0.10	181.81	0.31
30	10	20	162.08	0.04	150.8	0.03	156.4	0.03	153.56	0.08
		50	109.34	0.07	97.59	0.06	111.7	0.06	98.25	0.13
		100	70.77	0.13	72.97	0.10	84.75	0.10	75.54	0.22
40	10	20	124.49	0.04	125.7	0.03	134	0.03	122.02	0.08
		50	67.62	0.07	69.58	0.06	68.93	0.06	62.34	0.11
		100	40.34	0.14	43.53	0.10	43.81	0.10	39.25	0.17
20	15	20	131.13	0.04	117.3	0.03	160.2	0.03	111.96	0.09
		50	121.86	0.09	114.3	0.06	142.8	0.06	102.17	0.18
		100	137.94	0.16	150.2	0.11	176.2	0.12	124.24	0.34
30	15	20	118.78	0.05	128.8	0.03	116.1	0.04	110.92	0.10
		50	81.31	0.10	91.23	0.06	84.58	0.09	72.48	0.16
		100	60.42	0.15	70.56	0.11	63.79	0.12	55.38	0.25
40	15	20	78.65	0.04	90.41	0.03	107.4	0.03	78.19	0.09
		50	36.35	0.08	44.56	0.06	50.57	0.06	37.79	0.15
		100	19.79	0.14	27.31	0.11	27.8	0.12	23.55	0.24

In future, there will be further research in the following three directions: multi-objective scheduling problems, such as considering green and economic optimization simultaneously; building a cooperative multi-agent asynchronous decision-making model; improving the performance of the algorithm, such as introducing meta-learning to improve the generalization of the model.

Acknowledgements This work is funded by National key Research and development program of China (Grant No. 2020YFB1709800), Project No. 2022CDJSKJC20 supported by Fundamental Research Funds for Central Universities and project of science and technology research program of Chongqing Education Commission of China (Grant No. KJQN201900107).

Data availability The data that support the findings of this study are available from the corresponding author, upon reasonable request.

Declarations

Competing interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Akarsu, C. H., & Küçükdeniz, T. (2022). Job shop scheduling with genetic algorithm-based hyperheuristic approach. *International Advanced Researches and Engineering Journal, 6*(1), 16–25.

Aydin, M. E., & Oztemel, E. (2000). Dynamic job-shop scheduling using reinforcement learning agents. *Robotics and Autonomous Systems, 33*(2–3), 169–178. https://doi.org/10.1016/S0921-8890(00)00087-7

Blackstone, J. H., Phillips, D. T., & Hogg, G. L. (1982). A state-of-the-art survey of dispatching rules for manufacturing job shop operations. *International Journal of Production Research, 20*(1), 27–45. https://doi.org/10.1080/00207548208947745

Bouazza, W., Sallez, Y., & Beldjilali, B. (2017). A distributed approach solving partially flexible job-shop scheduling problem with a Q-learning effect. *IFAC Papersonline, 50*(1), 15890–15895. https://doi.org/10.1016/j.ifacol.2017.08.2354

Chang, J. R., Yu, D., Hu, Y., He, W. W., & Yu, H. Y. (2022). Deep reinforcement learning for dynamic flexible job shop scheduling with random job arrival. *Processes*. https://doi.org/10.3390/pr10040760

Duan, J. G., & Wang, J. H. (2022). Robust scheduling for flexible machining job shop subject to machine breakdowns and new job arrivals considering system reusability and task recurrence. *Expert Systems with Applications*. https://doi.org/10.1016/j.eswa.2022.117489

Gonzalez, T., & Sahni, S. (1978). Flowshop and jobshop schedules: complexity and approximation. *Operations Research, 26*(1), 36–52. https://doi.org/10.1287/opre.26.1.36

Han, B. A., & Yang, J. J. (2020). Research on adaptive job shop scheduling problems based on dueling double DQN. *IEEE Access, 8*, 186474–186495. https://doi.org/10.1109/Access.2020.3029868

Heess, N., Dhruva, T. B., Sriram, S., Lemmon, J., Merel, J., Wayne, G., Tassa, Y., Erez, T., Wang, Z., Ali Eslami, S. M., Riedmiller, M., Silver, D. (2017). Emergence of locomotion behaviours in rich environments. *arXiv preprint* arXiv:1707.02286.

Hermann, M., Pentek, T., & Otto, B. (2016). Design Principles for Industrie 4.0 Scenarios. *Proceedings of the 49th Annual Hawaii International Conference on System Sciences (Hicss 2016)*, 3928–3937. https://doi.org/10.1109/Hicss.2016.488

Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep Reinforcement Learning that Matters. *Thirty-Second Aaai Conference on Artificial Intelligence / Thirtieth Innovative Applications of Artificial Intelligence Conference /Eighth Aaai Symposium on Educational Advances in Artificial Intelligence* 3207–3214. Retrieved from: //WOS:000485488903036

Hunt, R., Johnston, M., & Zhang, M. (2014, 6–11 July 2014). *Evolving machine-specific dispatching rules for a two-machine job shop using genetic programming.* Paper presented at the 2014 IEEE Congress on Evolutionary Computation (CEC).

Jing, X., Yao, X. F., Liu, M., & Zhou, J. J. (2022). Multi-agent reinforcement learning based on graph convolutional network for flexible job shop scheduling. *Journal of Intelligent Manufacturing*. https://doi.org/10.1007/s10845-022-02037-5

Kuhnle, A., Kaiser, J.-P., Theiß, F., Stricker, N., & Lanza, G. (2021). Designing an adaptive production control system using reinforcement learning. *Journal of Intelligent Manufacturing, 32*(3), 855–876. https://doi.org/10.1007/s10845-020-01612-y

Lang, S., Behrendt, F., Lanzerath, N., Reggelin, T., & Müller, M. (2020, 14–18 Dec. 2020). *Integration of Deep Reinforcement Learning and Discrete-Event Simulation for Real-Time Scheduling of a Flexible Job Shop Production.* Paper presented at the 2020 Winter Simulation Conference (WSC).

Lei, K., Guo, P., Zhao, W., Wang, Y., Qian, L., Meng, X., & Tang, L. (2022). A multi-action deep reinforcement learning framework for flexible Job-shop scheduling problem. *Expert Systems with Applications, 205*, 117796. https://doi.org/10.1016/j.eswa.2022.117796

Li, Y. X., Gu, W. N., Yuan, M. H., & Tang, Y. M. (2022). Real-time data-driven dynamic scheduling for flexible job shop with insufficient transportation resources using hybrid deep Q network. *Robotics and Computer-Integrated Manufacturing*. https://doi.org/10.1016/j.rcim.2021.102283

Liu, R. K., Piplani, R., & Toro, C. (2022). Deep reinforcement learning for dynamic scheduling of a flexible job shop. *International Journal of Production Research, 60*(13), 4049–4069. https://doi.org/10.1080/00207543.2022.2058432

Luo, S. (2020). Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Applied Soft Computing*. https://doi.org/10.1016/j.asoc.2020.106208

Lv, Y., Li, C., Tang, Y., & Kou, Y. (2022). Toward energy-efficient rescheduling decision mechanisms for flexible job shop with dynamic events and alternative process plans. *IEEE Transactions on Automation Science and Engineering, 19*(4), 3259–3275. https://doi.org/10.1109/TASE.2021.3115821

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature, 518*(7540), 529–533. https://doi.org/10.1038/nature14236

Namjoshi, J., & Rawat, M. (2022). Role of smart manufacturing in industry 4.0. *Materials Today: Proceedings, 63*, 475–478. https://doi.org/10.1016/j.matpr.2022.03.620

Oh, S. H., Cho, Y. I., & Woo, J. H. (2022). Distributional reinforcement learning with the independent learners for flexible job shop scheduling problem with high variability. *Journal of Computational Design and Engineering, 9*(4), 1157–1174. https://doi.org/10.1093/jcde/qwac044

Ouelhadj, D., & Petrovic, S. (2008). A survey of dynamic scheduling in manufacturing systems. *Journal of Scheduling, 12*(4), 417–431.

Qi, Q. L., & Tao, F. (2018). Digital twin and big data towards smart manufacturing and Industry 4.0: 360 degree comparison. *IEEE Access, 6*, 3585–3593. https://doi.org/10.1109/Access.2018.2793265

Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2015). High-dimensional continuous control using generalized advantage estimation. arXiv:1506.02438. Retrieved from https://ui.adsabs.harvard.edu/abs/2015arXiv150602438S

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347. Retrieved from https://ui.adsabs.harvard.edu/abs/2017arXiv170706347S

Serrano-Ruiz, J. C., Mula, J., & Poler, R. (2022). Development of a multidimensional conceptual model for job shop smart manufacturing scheduling from the Industry 4.0 perspective. *Journal of Manufacturing Systems, 63*, 185–202. https://doi.org/10.1016/j.jmsy.2022.03.011

Shahrabi, J., Adibi, M. A., & Mahootchi, M. (2017). A reinforcement learning approach to parameter estimation in dynamic job shop scheduling. *Computers & Industrial Engineering, 110*, 75–82. https://doi.org/10.1016/j.cie.2017.05.026

Souza, R. L. C., Ghasemi, A., Saif, A., & Gharaei, A. (2022). Robust job-shop scheduling under deterministic and stochastic unavailability constraints due to preventive and corrective maintenance. *Computers & Industrial Engineering*. https://doi.org/10.1016/j.cie.2022.108130

Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (2000). Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems, 12*(12), 1057–1063.

Tao, F., Cheng, Y., Zhang, L., & Nee, A. Y. C. (2017). Advanced manufacturing systems: Socialization characteristics and trends. *Journal of Intelligent Manufacturing, 28*(5), 1079–1094. https://doi.org/10.1007/s10845-015-1042-8

Wang, X. H., Zhang, L., Lin, T. Y., Zhao, C., Wang, K. Y., & Chen, Z. (2022). Solving job scheduling problems in a resource preemption environment with multi-agent reinforcement learning. *Robotics and Computer-Integrated Manufacturing*. https://doi.org/10.1016/j.rcim.2022.102324

Wang, Y.-F. (2020). Adaptive job shop scheduling strategy based on weighted Q-learning algorithm. *Journal of Intelligent Manufacturing, 31*(2), 417–432. https://doi.org/10.1007/s10845-018-1454-3

Yan, Q., Wang, H. F., & Wu, F. (2022). Digital twin-enabled dynamic scheduling with preventive maintenance using a double-layer Q-learning algorithm. *Computers & Operations Research*. https://doi.org/10.1016/j.cor.2022.105823

Yang, S. L., & Xu, Z. G. (2022). Intelligent scheduling and reconfiguration via deep reinforcement learning in smart manufacturing. *International Journal of Production Research, 60*(16), 4936–4953. https://doi.org/10.1080/00207543.2021.1943037

Zhang, C., Song, W., Cao, Z., Zhang, J., Siew Tan, P., & Xu, C. (2020). Learning to Dispatch for Job Shop Scheduling via Deep Reinforcement Learning. arXiv:2010.12367. Retrieved from https://ui.adsabs.harvard.edu/abs/2020arXiv201012367Z

Zhang, L. X., Yang, C., Yan, Y., & Hu, Y. G. (2022a). Distributed real-time scheduling in cloud manufacturing by deep reinforcement learning. *IEEE Transactions on Industrial Informatics, 18*(12), 8999–9007. https://doi.org/10.1109/Tii.2022.3178410

Zhang, Y., Zhu, H., Tang, D., Zhou, T., & Gui, Y. (2022b). Dynamic job shop scheduling based on deep reinforcement learning for multi-agent manufacturing systems. *Robotics and Computer-Integrated Manufacturing, 78*, 102412. https://doi.org/10.1016/j.rcim.2022.102412

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.